FlagPrism: Compiler-Assisted Fine-Grained Profiling Across AI Accelerators (Operational Systems)

Anonymous Authors

Abstract

Porting an optimized operator across AI accelerators preserves its mathematics but can change its execution organization and the meaning of profiling data. We present FlagPrism, a compiler-assisted profiler for selective offline operator diagnosis on NVIDIA, GCU, and Ascend. Its execution manifests, typed dynamic observations, and invocation reconstruction preserve backend detail while exposing incomplete attribution and unverified clocks. Two retrospective workflows connect execution evidence to independently validated configuration changes, with 1.62× and two-stage 5.31× recovery. Implementation experience exposes incomplete logger publication under vector ownership, ambiguous SDK dispatch/compute views, and local waits that change by 77–83% between observation modes despite low whole-task perturbation. These results support qualified structural and qualitative diagnosis, rather than precise uninstrumented wait costs or a certified shared pipeline timeline.

1 Introduction

Unified kernel frameworks let operator developers reuse program logic across AI accelerators. For example, FlagTree extends Triton to multiple backends [1]. We consider NVIDIA as the optimization starting point: developers port implementations and configurations tuned there to other accelerators. Performance still depends on how the target maps computation, data movement and synchronization. Profiling must connect those execution observations to the implementation being optimized.

Architecture diversity appears at several levels. GPUs with similar SIMT models can differ in thread organization, local memory, resources and asynchronous facilities [6, 7]. Dedicated compute and data-movement pipelines add different scheduling and synchronization rules [4]. Compilation can split, fuse, reorder or pipeline a source region; observing an asynchronous issue need not measure completion. A common interface alone therefore does not determine which events to observe or how to relate them back to the program.

Existing work supplies important foundations. KPerfIR integrates fine-grained profiling into compiler intermediate representation (IR) and analyzes intra-kernel execution and overlap [2]; Neutrino supports programmable instruction-level probes [3]. Proton combines vendor measurement interfaces and instrumentation with multi-level program context [9], while PASTA provides a modular analysis framework for accelerator workloads [5]. These capabilities motivate a concrete question for a broader collection of execution backends: which program mappings, instrumentation semantics, and measurement assumptions can be reused, and which must change when the target architecture changes? The study uses their NVIDIA information as its diagnostic reference, while our closest-prior comparison includes KPerfIR's AMD publication and asynchronous-wait replay mechanisms. The gap studied here concerns concrete additional compiler/SDK boundaries, not an absence of fine-grained profiling or cross-vendor support in prior work.

This paper investigates selective offline profiling for diagnosing and recovering performance lost after porting. Structural mappings and qualified instrumented observations support decisions; precise uninstrumented local costs and a certified shared pipeline timeline remain outside our service. The resulting observations must explain the logical operator region or execution behavior being optimized. We examine three sources of difficulty. First, compilation changes the relationship between logical regions and execution instances. Second, instrumentation must respect the target's execution semantics, including the distinction between submission, execution, and waiting. Third, timestamps, recording mechanisms, and vendor observations must be reconstructed into program-associated results without hiding architecture-specific detail. The study examines targets with similar and different execution models.

We place an empirical study before the system design. Starting from NVIDIA-tuned operators, it examines performance after porting and the portion recoverable through target-specific changes. Two cases connect execution-level causes to fine-grained diagnosis, implementation changes, and unin-

strumented performance recovery. They expose information that ordinary timing and source-region observations leave unresolved. The first case keeps a cooperative row-program organization while NVIDIA warps and GCU vector work-items receive different mappings; the second unfolds one fused tile into Ascend AIC/AIV producer–consumer actors. We do not classify GCU as NVIDIA-like SIMT hardware. Target-local configuration changes recover 1.62× and two-stage 5.31× performance in the studied cases. The final public service retrospectively reproduces the mapping clue and first producer-granularity decision; the second fusion step uses instance cardinality and independently measured task recovery, rather than lower waits.

We investigate FlagPrism, a profiler whose system boundary includes the profiling-related compilation mechanisms in FlagTree, together with runtime collection, event association, and analysis. Its service is to lower common profiling requests into target-appropriate observations and reconnect those observations to the operator program. It serves operator developers and optimization tools through three mechanisms. A compiled execution manifest retains specialization, actor, and surviving source relationships. Execution-aware instrumentation produces typed dynamic observations with explicit selection and recording limits. Evidence-based reconstruction binds observations to actual invocations and preserves clock and attribution boundaries. The implementation extends existing compiler passes, metadata, and collectors; it does not require a new programming language. Missing source origins and unverified cross-actor clock relations remain explicit.

The added public knowledge is the demonstrated trust failures, their concrete implementation responses, and the diagnostic decisions that survive qualification. The paper develops the following contributions:

- An empirical study of performance loss after porting NVIDIA-tuned operators, with execution-level diagnoses and targeted recovery under cooperative program and producer–consumer organizations (Section 3).
- A compiler–profiler implementation combining execution manifests, typed dynamic instrumentation, and evidence-based invocation reconstruction across NVIDIA, GCU, and Ascend backends (Section 4). The implementation addresses concrete boundaries: ownership-safe record publication, typed AIC/AIV readiness, and retained-worker association through SDK dispatch and compute contexts.
- Hardware-qualified validation of mathematical correctness, dynamic recording, invocation association, and measurement costs on three backends (Section 6), including counterexamples that distinguish complete publication from correct math, canonical compute tasks from SDK context views, and local wait stability from whole-task perturbation and end-to-end cost.

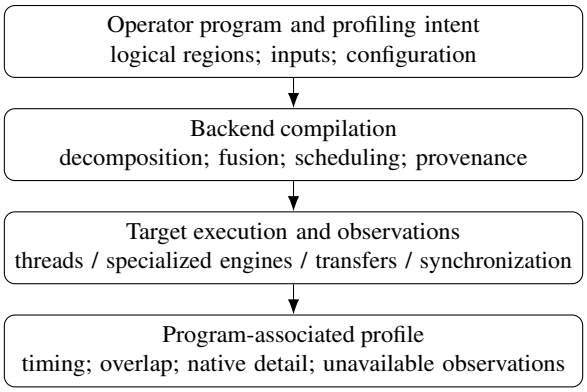


Figure 1: Conceptual profiling scope. Branches illustrate possible mappings, not measured schedules. The study supplies concrete compiled examples.

The evaluation concerns three research environments and controlled operator experiments; it does not establish production deployment or broad adoption.

2 Background and Profiling Scope

2.1 Operators, Backends, and Execution Models

We study Triton operator programs compiled through FlagTree. Shapes, tiling, worker requests and pipeline parameters specialize a program into a binary; a portable parameter name need not select equivalent hardware organization. The tested devices are an NVIDIA RTX PRO 6000 Blackwell, an Enflame ZIXIAOC200 GCU, and an Ascend 910B4-1. Their pinned compiler and SDK stacks form part of the experimental target. We distinguish NVIDIA SIMT warps, GCU vector work-items, and Ascend AIC/AIV producer–consumer actors; a vendor label or the term GPU does not establish an identical execution model.

2.2 From Program Regions to Device Execution

A logical region is a developer-visible part of an operator. A compiled site belongs to one binary, while a dynamic occurrence also belongs to one launch, program and actor. Lowering can split, fuse or duplicate regions; a surviving location may identify several candidate origins. Static ownership and compiler dependencies describe organization, not measured utilization or cost. We preserve these distinctions rather than requiring one source operation per hardware event.

2.3 Measurement Semantics

Host submission, native device-task execution, source-control brackets and engine activity have different scopes. In particular, issuing an asynchronous operation does not establish its completion; a wait bracket can change when instrumentation redistributes work. Counters also require separate evidence for their rate and epoch. Unknown-frequency GCU ticks remain ticks; supported Ascend source counters provide actor-local durations without a certified shared source/native origin. Independent SDK replays retain their own capture identities and overlapping activities are not summed as exclusive wall time.

3 Study: Diagnosing Performance Loss after Porting

Story. NVIDIA-tuned implementation → correct target port with different performance → execution-level cause → backend-specific diagnosis → targeted change and recovery.
Setup. Six operators; three shapes and two frozen NVIDIA configurations per shape; RTX PRO 6000, ZIXIAOC200, and Ascend 910B4-1. The source implements FP16-input/FP32-accumulation GEMM and fused GEMM+bias+GELU; softmax, layer normalization, reduction, and transpose use FP32. Identical input bytes → CPU-reference correctness checks → saved compiled mappings → native kernel/task intervals. All execution, tuning, and collection run in server Docker containers. Backend compiler/SDK versions differ; clocks and exclusive device use are not fixed. Results characterize these implementations and software stacks. Uninstrumented and recovery timings below disable source-level observers; the native activity collector remains enabled to identify kernel/task intervals. They are not measurements with every profiling facility disabled.
Optimization starting point. A bounded NVIDIA search covers 1,152 configurations across 20 operator/shape combinations, including the two long-K matrix cases; 1,112 pass and 40 exceed shared-memory limits. Resident CUDA-graph throughput → shortlist three distinct binaries → three native batches of 30 calls per candidate → freeze the native-selected configuration. The scope includes GEMM warp requests 4/8 and row/transpose requests 1/2/4/8; this is not exhaustive or vendor-best tuning. Independent post-freeze captures supply the plotted performance distributions and case comparisons.

3.1 Performance Differences across an Operator Set

Coverage. Compute, fusion, reductions, and data movement → different shapes and source configurations → Figure 2. Configuration0 is the frozen NVIDIA selection; configuration 1 is the next distinct native-shortlisted configuration. Ports retain the requested configurations and mathematical operations.

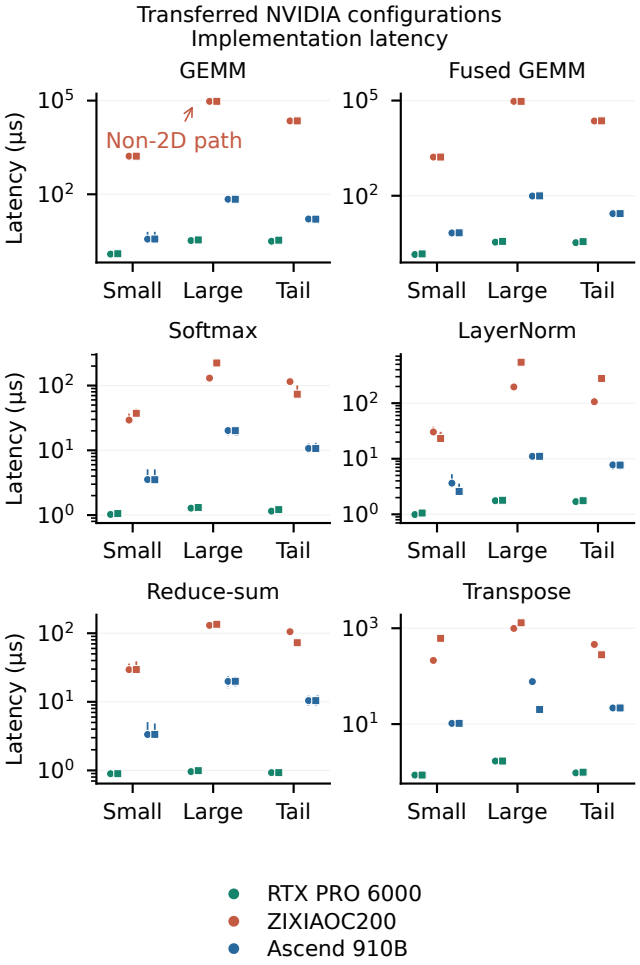


Figure 2: Transferred NVIDIA configurations: native implementation latencies of the ported implementations. Each shape/configuration has three batches of 30 prewarmed calls per backend. Small dots: calls; circle/square: configuration 0/1 medians; vertical spans: descriptive min–max, not confidence intervals. Matrix shapes are 128^3, 512^3, $257 \times 193 \times 320$; row shapes 32×256, 256×1024, 127×769; transpose shapes 256^2, 1024^2, 513×257. Configurations are selected on NVIDIA and requested unchanged on targets. Large GCU matrix values reflect the implementation path discussed in the text; this is not a hardware-peak ranking.

Each backend contributes 108 captures and 3,240 positive native intervals; all 324 captures pass input/configuration and numerical checks.
Interpreting large gaps. Duration alone does not distinguish hardware capability from an inefficient implementation path. For example, GCU GEMM 512^3 with the transferred 32^3 tile and four work-items takes approximately 94.74 ms; an independent device-stream measurement gives 94.76 ms. The

compiled per-item matrix is 16×16, with zero reported 2D instruction share. At the same grid, one work-item produces a 32×32 local matrix, positive 2D share, and 268.53μs. This sanity check identifies a software-path contribution; it is not a chip-peak comparison or a universal tile threshold. A 16×32 control also reports 2D activity, so local shape alone is not a universal threshold. This check qualifies the overview; the two main cases below provide more direct program-level diagnoses and controlled adjustments.

3.2 Case A: Cooperative Reduction Mapping

Scope. One row program and cooperating execution groups are common to both ports; NVIDIA warps and GCU vector work-items have different capacities and instruction organization. We compare this shared program organization, without classifying ZIXIAOC200 as NVIDIA-like SIMT hardware.
Observation. Softmax 127×769, one row per program. NVIDIA selects four warps: independent native medians are 1.152μs for four and 1.248μs for one. The exact GCU port instead improves from approximately 115.06 to 71.12μs when the corresponding request changes four work-items to one, with grid 127 and the input fixed: 1.62× target-local speedup.
Diagnosis. Timing ranks configurations but does not explain how cooperation is organized on the target. Actual lowering exposes ownership and partial exchange; this suggests testing fewer vector work-items, with the prediction that a coarser per-item reduction can improve the port. Same request → different ownership. Actual NVIDIA indexing stripes columns across warps, assigning 193/192/192/192 useful elements; GCU assigns contiguous slices of 256/256/256/1. GCU four-item lowering uses shared partial exchange and two barriers; one-item lowering owns the full row and removes the exchange. Original-binary replay counters show the SDK 1D instruction-cycle share rising from 0.841% to 2.991%. The share uses SDK total cycles as its denominator; it is a companion execution characteristic, not GPU occupancy or an isolated explanation of the speedup. Static element counts do not measure utilization or prove padding as the sole cause.
Controlled adjustment. Hold source math, input, row grouping, and grid fixed → change only the worker request → verify the changed mapping and three independent 30-call native batches. A two-item control keeps both barriers but reaches approximately 73.65μs; barrier presence alone therefore does not explain the four-item slowdown. Ownership, local/shared resources, and worker-dependent synchronization costs change together. The adjustment is validated; their individual cost contributions remain unseparated.

3.3 Case B: Producer–Consumer Granularity

Observation. Fused GEMM $512 \times 512 \times 4096$, a longer-$K$ shape outside the overview, exposes producer–consumer behavior. The frozen NVIDIA implementation uses 32^3 tiles,

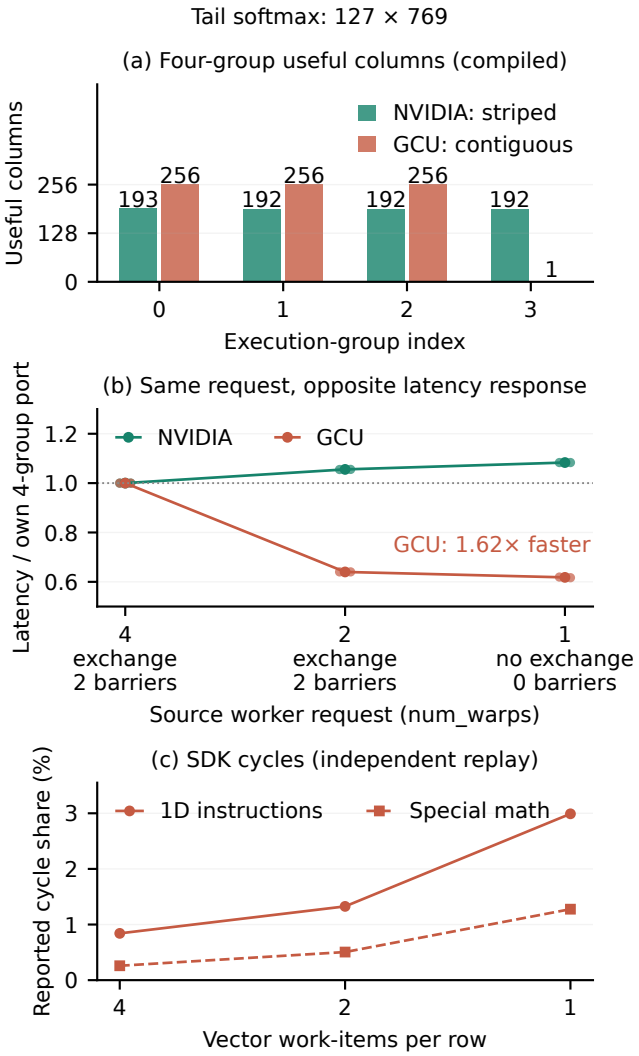


Figure 3: Tail softmax. (a) Compiler-verified useful-column assignment to 32-lane NVIDIA warps versus GCU vector work-items; capacities differ. (b) Uninstrumented latency normalized to each backend's four-group version; dots denote three batch medians, 30 calls per batch. (c) SDK-defined per-SIP cycle shares from original-binary TCU replays, 21 calls including six initial/warmup calls, 24 SIP instances per record. Replay counters are separate captures and are not confidence intervals or exclusive wall-time fractions. The two-item control shows that barrier presence alone does not explain the slowdown; synchronization cost itself is not isolated.

four warps, and stage request 4; independent native latency is 17.728μs. Its Ascend port takes 310.27μs. Each logical tile becomes one AIC producer and two AIV consumers, connected by a global-memory workspace and readiness event 0. The stage request enables the backend's multibuffering; it does not mean four physical buffers.
Diagnosis. Long output-stage time → computation or readi-

ness? In SDK program 0 replays, AIV0 wait0 activity is 46.72μs while vector activity is 0.293μs. Increasing only B_K from 32 to 256 leaves the saved AIV LLVM byte-identical, but reduces producer Cube commands 128→16, MTE2 activity 47.52→23.81μs, and AIV0 wait0 46.72→26.40μs; vector activity stays 0.293μs. These activities may overlap and are not added to reconstruct wall time. Sparse scalar-clock records expose iteration IDs, issue gates, and L1 buffers. Within the retained study campaign, all three program 0 actor windows have one compatible native-window match under a shared reported-counter assumption. This is a campaign-level consistency check, not an independent runtime-identity or counter-domain certificate. Figure 4(b) conditionally places the measured producer control and consumer wait brackets in that frame; bars measure source control, not engine occupancy. Program 0 is one logical tile with three actors; these local observations diagnose a dependency, while whole-launch time covers all programs. They neither represent every instance nor reconstruct a launch critical path.

Reusable service boundary. The study's sparse research probes supply richer iteration/buffer details than the final public service. The latter reproduces typed static readiness, local waits and producer counts, without the conditional absolute placement. All-site versus wait-only collection changes the original local waits by 77–83%, while preserving the first intervention's qualitative direction (Table 1). The second recovery is supported by cardinality/loading evidence and independent native timing, not by a stable further wait reduction (Table 2).

Staged recovery. Producer granularity first: $32 \times 32 \times 32 \rightarrow 32 \times 32 \times 256 \rightarrow$ 310.27→164.12μs, with identical output tiling and logical input payload. The compiled instance and source-loading counts suggest output tiling next: $64 \times 64 \times 256 \rightarrow$58.45$\mu$s. Logical programs 256→64 and source-requested A/B payload 128→64 MiB; physical DRAM traffic is not inferred from this count. The larger-tile case does not materially lower program 0 wait further: the second gain comes with reduced instance cardinality and source-requested loading; their separate causal contributions are not identified. Held-out randomized native batches and CPU checks confirm the two-step 5.31× recovery. Additional B_K and unroll explorations, including non-improvements, remain in the artifact with their distinct configurations; they are not all controls of the final frozen implementation.

3.4 Information Needed to Diagnose the Loss

Case A. Total latency shows a target-side gap; source-level grouping alone does not reveal striped versus contiguous ownership, shared exchange, or vector resource activity. Compiled mappings → fewer-item hypothesis → controlled worker adjustment; per-SIP counters characterize the changed path.

Case B. A long output region does not identify the activity responsible. Actor identity + readiness event + producer com-

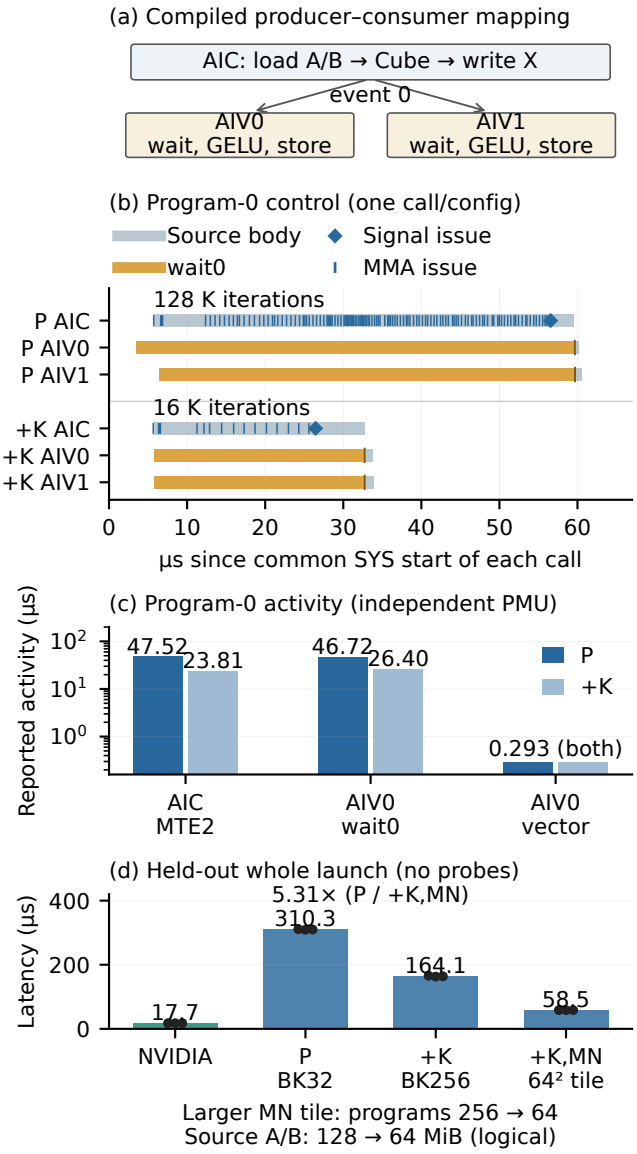


Figure 4: Fused-case diagnosis and recovery. P: NVIDIA port; +K: $B_K = 256$; +K,MN: also $B_M/B_N = 64$. (a) Compiled dependencies. (b) Program 0, warm-up 0 SDK replay. Placement assumes a shared reported SYS frame; window consistency does not certify counter-domain identity. Gray: source body; gold: scalar wait0; ticks: MMA-helper issue; diamond: signal issue. These are control positions, not engine busy spans. (c) Independent PMU replays; activities overlap and conditions differ from (b). (d) Independent native latency, 90 calls/configuration; dots: three batch medians. The public service does not certify this shared placement or recover these bespoke dynamic buffer identities. Sparse-control whole-task ratios 1.0045/0.9975/1.0071 describe separate captures, not zero-overhead guarantees for (b).

mand granularity → distinguish waiting from vector execution → improve the producer before changing the consumer.
Profiling gap. Proton and KPerfIR supply NVIDIA profiling information, including fine-grained regions [2, 9]. That information does not directly specify GCU vector ownership or Ascend AIC/AIV readiness dependencies. Explaining these cases requires backend-specific execution information associated with the source operator, beyond ordinary kernel or region durations. Timing-only search can still find faster configurations; the demonstrated gap is diagnostic explanation, not an impossibility of optimization without us. Our generic probes are not executions of those tools.
Transition to the design. Source operator/region → actual execution instances and native activities → optimization feedback. This motivates profiling services in FlagPrism with compilation support from FlagTree: preserve backend detail and its program association. The study validates research observations and configuration changes; a general profiling IR or production mapping mechanism must be substantiated separately in the design and evaluation.

4 Compiler-Assisted Profiling Design

FlagPrism provides profiling feedback for operator optimization. A request specifies observation semantics, programs, sites, capacity, and occurrence sampling. Each backend produces the requested observations with explicit measurement conditions or rejects unsupported requests. A common request does not require different execution models to expose identical actors or clocks. The design combines a compiled execution manifest, execution-aware instrumentation, and evidence-based association and reconstruction. The cooperative case requires ownership information both to explain the operator mapping and to publish complete observation records. The fused case requires a readiness channel, separate producer and consumer observations, and a binding from those actors to the source invocation. These requirements connect the three mechanisms; an interface that merely renames counters does not supply the missing relationships.

4.1 Mapping the Compiled Execution

Each actual launch links its arguments and configuration to a compiled binary. Its execution manifest retains IR, backend organization, observable sites, and surviving compiler-assigned operation or scope identifiers. Site IDs are qualified by the binary; an ID in another specialization is not the same site.

Fusion and lowering can associate one operation with several sites, or retain several source origins for one site. Adapters preserve available relationships and distinguish unique, ambiguous, and missing origins. A compiler location identifies an origin candidate, not exclusive cost ownership. An enclosing region's duration cannot be independently assigned in full to each operation. Backend actors and available dependency or buffer information remain explicit; static relationships are not measurements of when an actor ran. Runtime dispatches, device compute contexts, and compiler actors are distinct levels. In the observed Ascend mixed execution, one dispatch has an SDK dispatch-envelope view and a compute-context view; that compute context uses both AIC and AIV actors. Treating both SDK views as independent kernels would double count the execution. The importer follows SDK context membership and the compute-task table's explicit task reference. It associates a source invocation only when the dispatch has one uniquely identified compute member, preserving its other context views as dispatch evidence. Multiple compute members remain explicit without assigning the source invocation to an arbitrary member. A context number alone does not determine whether a task performs computation. For Ascend, the adapter captures typed synchronization IR after the SDK's graph synchronization solver, alongside the original and instrumented actor LLVM from the same compilation. A synchronization site records its emitting core kind, producer and consumer pipelines, and event flag. The adapter first validates exact retained source and manifest bytes against their hashes, the successful enclosing and SDK compilation identities, and the actor LLVM artifact and completed driver job. Parsed manifest values must also agree, including their types. Readiness and source-origin imports share this validation; missing or stale evidence retains raw inventory without asserting a verified association. The adapter then requires a unique corresponding actor in that compilation's LLVM inventory. It groups verified sites by the typed event channel and creates an edge only for a unique producer and consumer in different actor roles. Missing or multiple candidates remain partial. Unresolved typed sites still count as competing channel candidates; discarding their rejected or missing artifacts cannot certify uniqueness. This recovers AIC-to-AIV readiness without guessing private LLVM operand encodings. It is a compiler-static dependency, not a pairing of dynamic flag generations.

With line information enabled, surviving LLVM locations are matched against the retained source-operation snapshot. Exact locations can still identify several origin candidates after lowering. The fused case validates the typed producer–consumer edge and surviving site locations; its explicit source operation IDs remain absent. The manifest consequently preserves both origin candidates and unresolved associations instead of assigning exclusive costs.

4.2 Observing Dynamic Execution

Observations carry semantics, site, program, actor, occurrence, clock scope, and collection provenance. A source-control interval measures execution between instrumented control points, whose cost may differ from the uninstrumented execution; an issue interval brackets submission; a wait bracket includes control waiting; a checkpoint records a point. These

are not automatically completion times or engine occupancy. Instrumentation does not add a completion barrier to manufacture a sequential pipeline.

Repeated visits append dynamic records rather than overwrite one static slot. Backend-specific sequence and attempted-occurrence fields expose sampling and capacity losses where supported. The decoder validates each publication protocol; occurrence numbers are not assumed to be source-loop indices. Snapshot and dynamic layouts are validated separately, retaining saturation and incomplete-record reasons.

NVIDIA observes explicit profiling scopes with device program selection in the supported single-CTA configuration; output site filtering does not reduce its device instrumentation. GCU uses one selected program and a sole work-item publisher. Its bounded-index implementation uses 32-bit trace arithmetic while retaining the full SDK counter words, avoiding a global 64-bit arithmetic change to the reference kernel. Publisher selection must also respect the lowered value layout: selecting work-item zero does not guarantee that it owns every word of a vector record. The default emits scalar words under the selected publisher. The vector alternative repeats a complete record in each work-item's ownership slice, then publishes from the selected work-item; it does not partition a record across inactive publishers. This alternative addresses SDK compilation feasibility, and its mode and actual configuration enter the cache key. It does not imply lower perturbation. The final record tag is validated after capture synchronization, not used as a concurrent release/acquire protocol.

Ascend prepares a hidden observer argument before normal lowering and places role-specific observations in late SDK LLVM. The runtime owns trace storage. The allocation contract validates the final three-actor layout before any device allocation. Formal hidden-only row and fused matrix–bias–GELU captures preserve numerical results and produce dynamic records through this path. The fused capture observes one AIC and two AIV actors for one selected logical tile. Requested semantic classes are selected in the late pass before insertion: unselected primitive brackets and loop checkpoints receive neither counter reads nor records. Selected sites retain their ordinals from the unfiltered enumeration of the same original actor IR. The effective selection enters compilation identity and the manifest. This is separate from the program predicate, which limits record publication; counter reads at selected sites can still execute in other programs. Output filtering retains the request boundary but does not itself establish lower cost. Arbitrary site selection and occurrence sampling remain unsupported on this path. The six-family operator matrix additionally checks the complete path on 36 detailed configurations. Held-out RMSNorm checks varying shapes and configurations; a long-K fusion validates the same mechanism after tile and reduction-step changes. These checks establish the tested recording path, not universal event coverage.

4.3 Associating and Reconstructing Feedback

Source mapping, invocation identity, and clock alignment are separate obligations. Launch metadata identifies a source invocation and its actual specialization. Native activity requires an independent invocation association; grouping does not itself align timestamps.

On Ascend, the caller and SDK launch worker can differ. A retained per-launch token therefore crosses a versioned capsule into a pure C++ worker callback. Immediately after the original launch returns, the actual worker queries its device, stream, and last task ID and publishes the original statuses with its process and thread IDs. The callback needs neither Python's GIL nor an added completion synchronization. Cancellation, duplicate publication, late callbacks, and closed epochs remain explicit; zero task IDs are valid when their queries succeed.

The collector seals capture lifetime through a unique newly created SDK directory and matching process/device metadata. The importer requires a unique native task and host-launch connection tuple within that capture. It does not join by names, order, or timestamp containment. Multiple source receipts naming one task remain ambiguous. Four actual calls across two binaries and two streams validate the retained worker and SDK connection-table path, including task zero. The default row-kernel capture retains the complete SDK output and associates four native tasks with their source invocations. Native copy, marker, and synchronization records retain their own activity kinds rather than entering the kernel hotspot table. A fresh fused capture additionally validates four source-associated compute tasks while retaining their dispatch context views.

Clock descriptors retain scope, counter width, conversion evidence, and alignment status. Instance-local ticks remain local without shared-domain evidence; unknown frequency does not produce nanoseconds. Reconstruction never stretches internal intervals to a native task envelope. That envelope does not establish per-program occupancy or a critical path. Independent counter and replay captures retain separate identities.

Rate conversion and epoch alignment are separate. For the supported Ascend A2 path, local conversion requires the exact source invocation and binary, the retained device frequency, and matching compiled and SDK system-counter instructions. A successful conversion supplies source-control durations while preserving the actor-local clock. The raw SDK task-counter producer retains packet bytes, offsets, and identities independently; matching a task does not establish a common source-clock offset. On this path one counter tick is 20 ns; a zero-tick bracket does not establish zero execution cost. The present integration therefore exports these source observations without a certified shared source/ACSQ origin. Native association and local duration conversion remain useful, but do not authorize a cross-actor critical path.

Outputs preserve raw records, compiler artifacts, partial re-

lationships, and unavailable-quantity reasons alongside the reconstructed view. Developers and tuning systems can inspect backend extensions without treating missing information as a measured zero.

5 Implementation

FlagPrism extends FlagTree's compiler hooks and the FlagPrism native collectors. The shared Python layer validates observation requests, records launch and binary provenance, exports typed observations, and reconstructs associated views. Compiler adapters extract execution manifests; backend passes and launchers implement the supported observation semantics. Vendor collectors retain native task and counter information as separate evidence. The implementation inherits frontend launch metadata, explicit profiling scopes, compiler pass infrastructure, and vendor collectors. It adds backend execution adapters, the hidden-argument and late-LLVM observation path, publication contracts, retained-worker identity and SDK-context validation, and provenance-preserving reconstruction. The tested FlagTree baselines are `57d348` (NVIDIA), `add842f2` (GCU), and `0364ed4b` (Ascend), with backend integration patches retained separately. The final shared runtime is `8fffdf6`; `1cb15d7` identifies an earlier qualification snapshot. Backend manifests pin actual libraries, integration patches and SDK dependencies. Full hardware matrices retain their earlier campaign versions; the final provenance repair has a fresh four-launch fusion check and exact-source Docker regression, not relabeled matrix timing.

5.1 Backend Integration and Capability Status

The NVIDIA implementation uses explicit Triton scopes and CUPTI native activity. GCU combines compiler-assigned sites with the SDK global timer and TOPSPTI tasks. Both have actual dynamic-record and operator-correctness tests. The Ascend integration combines hidden observer storage, late LLVM role/site instrumentation, native ACL profiling, and retained worker receipts. Its generated worker and native task identity path pass four actual calls across two binaries and two streams. A hidden-only row-kernel smoke preserves numerical results and produces dynamic observations; local source durations are separately validated against retained compiler and SDK evidence. The default row-kernel export retains SDK files and 32 local-duration records for four associated tasks. The fresh fused capture validates four tasks and 232 records across one AIC and two AIV actors. Both preserve actor-local scope without a verified shared epoch. Section 6 reports the six-family, held-out and long-K hardware campaigns with their own versions; CPU contracts do not replace those device tests. The final Ascend wheel preserves the validated compiler and two native libraries, all 52 component Python modules, and the receipt ABI header; isolated public imports and compiler/allocation guards pass. This is packaging of accepted binaries, not a separate clean-source build.

5.2 Limitations and Failure Handling

Unsupported selection or semantic requests fail explicitly. Available source origins can remain ambiguous, and unknown-frequency or local counters are retained without fabricated global timestamps. The capture bundle includes raw observations, compiler artifacts, native outputs, and file hashes. Partial captures and original application errors survive cleanup failures. Native observation export retains a recovery file before acknowledging queued captures; atomic sidecar replacement avoids losing raw records on write errors. SDK output cleanup follows successful copying and bundle export, preserving the original session directory when export fails.

Native start failure rolls back session activation and collector ownership so the caller can retry. Receipt contexts retain queued-worker lifetime and reject late publication after capture closure. Independent replay captures are not joined to an earlier invocation; graph, concurrency, and counter capabilities remain backend-specific. Detailed NVIDIA scopes require annotation and the supported single-CTA configuration; device occurrence sampling is unavailable. GCU timestamps have no certified frequency conversion. Ascend accepts one selected logical program and compile-time semantic classes, but rejects arbitrary site selection and occurrence sampling. Its source-to-ACSQ epoch relation remains unverified. These restrictions are exposed rather than hidden by the common request vocabulary.

Starting with a bounded request. On NVIDIA, begin with an annotated region, program 0, a bounded buffer and the supported single-CTA configuration; inspect attempted/dropped records before interpreting the scope. On GCU, inspect the binary-specific tracked operation inventory, select one store and program 0, and use scalar publication with capacity 64. The tested Softmax request completes; arbitrary all-site compilation is not an established operating envelope. On Ascend, select one logical program and existing wait semantics; reject unsupported arbitrary sites or sampling. Check provenance, record loss and local clock status before comparing an intervention. These are offline diagnostic requests, with the separate compilation, device, host and export costs in Section 6.

6 System Evaluation

The study in Section 3 characterizes the problem. We distinguish functional regression on its operator families from held-out validation and mechanism-specific tests. Reusing a study family for correctness does not establish generalization to unseen operators. RMSNorm was selected for held-out validation before the final experiments. The final shared-source

regression runs inside each server Docker; code and native-library hashes qualify results from earlier hardware matrices. Additional devices remain outside this evaluation.

6.1 Measurement and Attribution Fidelity

The GCU functional matrix covers six operator families, three shapes, and two configurations, including tails. Its explicit vector-publication mode passes all 36 cases with fixed inputs, original CPU-reference tolerances, input immutability checks, and output canaries. Across 108 actual task captures, the bounded all-site request retains 4,599 observations and declares 21,996 capacity-dropped records. These are bounded traces, not complete execution histories. An independent artifact audit checks the 108 raw-buffer hashes and the recorded case and request identities. GCU timestamps remain raw ticks; this experiment does not certify frequency or cross-actor clock alignment. The default scalar all-site path completed six matrix cases before a GELU compilation reached the 900 s limit. The explicit-vector matrix completed in 70.85 minutes. The latter establishes compilation feasibility for that mode, not practical all-site collection cost or acceptance of the default path.

The Ascend matrix executes the same six families at three shapes and two configurations in off, native, and detailed modes, for 108 requests. Numerical results, input immutability, and output canaries pass in each mode. The 36 detailed requests produce 7,788 records from 216 per-invocation raw buffers, with no declared capacity drops or counter saturation. All retained records have local duration conversion and preserve instance-local clock scope. Exact native task association is validated for the 72 native and detailed requests; off-mode task identity is unavailable. These results establish recording and association at the supported scope, not complete source-operation attribution or a shared actor epoch.

Held-out RMSNorm adds three shapes and two configurations in all three modes; all 18 requests pass the same numerical, input, and canary checks. The 12 native and detailed requests have exact task association, and each of the six detailed requests produces observations. This is held-out coverage of one family, not arbitrary-kernel support. The frozen shared-source regression passes 491 tests on NVIDIA, 414 on GCU, and 477 on Ascend; respectively 4, 5, and 20 tests are explicitly skipped for opt-in browser or backend/fixture requirements. Focused hardware checks on the preceding source include three NVIDIA selected-program requests and six GCU calls across two streams and two specializations. The final provenance repair adds a fresh four-launch Ascend fusion capture: numerical/input/canary checks pass, and 34 unique source-location matches, 27 ambiguous and 13 unassociated origins remain explicit. Earlier operator matrices retain their own version manifests. These checks establish functional regression, not independent diagnostic timing accuracy.

Tile (B_M, B_N, B_K)	All sites	Wait only
$(32, 32, 32)$	55.37/54.72	31.31/29.83
$(32, 32, 256)$	13.74/12.74	14.22/12.07
$(64, 64, 256)$	14.44/12.66	13.00/11.99

Table 1: Actor-local readiness-wait medians in μs, program 0. Each cell lists the two AIV actors separately; ten measured calls per mode/configuration. These are instrumented control intervals, not uninstrumented costs or aligned engine activity.

Diagnostic stability is a separate test. We collect the three fusion configurations of Section 3.3 with all supported sites and with compiler-selected wait sites only, using a fixed shuffled order, three warmup calls and ten measured calls per request. Every request passes the unchanged numerical/input/canary oracles and exact task association. All-site AIC signal sites each execute 128 times at $B_K = 32$ and 16 times at $B_K = 256$; loop-header visits are respectively 129 and 17, including the terminating visit. These counts agree with the original LLVM control flow and configuration, without treating occurrences as induction variables. Each AIV readiness site executes once per call in both modes.

Table 1 exposes a substantial negative result: the original configuration's all-site waits exceed wait-only medians by 76.8% and 83.4%. We therefore do not use the earlier low whole-task perturbation result as a certificate of local wait accuracy. Both modes preserve the reduction when B_K increases; the smaller difference between the last two configurations is not stable. We use the former qualitative decision and refuse to attribute the second speedup to a further reduction in waiting. Independent SDK replay activity also decreases for the first intervention, at its own scope and source version; its values are not interchangeable with these control brackets. A separate minimal two-read source control produces 64 zero-tick spans with distinct advancing start counters across calls. This control makes the resolution limit visible; it does not certify adjacent machine instructions or bound perturbation of the fusion's waits.

6.2 Overhead and Execution Perturbation

In a controlled GCU Softmax experiment observing one selected store site, three batches of 30 calls show 3.08–3.57% median device-task perturbation with scalar publication. The vector alternative incurs 26.47–26.95% under the same request. Scalar publication therefore remains the default; vector publication provides an explicit alternative for SDK compilation feasibility. Neither result establishes all-site cost. Detailed host invocation latency is about 5.2 ms, substantially exceeding native submission, so the device-task result does not imply low end-to-end overhead.

Ascend's all-site request is substantially more intrusive on the held-out RMSNorm shape 127×769. Across three

batches of 30 calls per mode, native task medians are 6.33–6.60 μs and detailed medians are 10.49–10.64 μs, a paired increase of 61.2–67.6%. Detailed host invocation latency is 1.79–1.81 ms, compared with 218–222 μs for native collection and 79–85 μs off. The off phase has no isolated native task reference, so it cannot establish native collection's device-task perturbation. The detailed result exceeds our engineering target; bounded output and one selected program do not establish low perturbation. Host invocation latency includes detailed capture's synchronization and record handling; final bundle export is reported separately.

Selecting only wait semantics in the compiler halves retained RMSNorm events from twelve to six per call, with no counter reads at unselected sites. However, three new 30-call batches still show 60.1–78.0% device-task perturbation and 1.71–1.76 ms host invocation latency. Reducing event count therefore does not establish a practical low-perturbation mode for this small kernel. The profiler exposes this cost boundary rather than presenting the reduced request as a performance improvement.

The preselected long-K fusion has a different cost regime. For its original 32^3 output/reduction tile request at $512 \times 512 \times 4096$, three batches of 30 calls give native task medians of 309.74–311.19 μs and all-site medians of 310.94–313.76 μs: paired changes of -0.08% to $+1.30\%$. This meets the device-task target for that configuration only. Detailed host invocation latency is 12.33–12.46 ms versus 267–272 μs natively; detailed export takes 15.62–17.21 s and produces 221–229 MB per session of 33 calls including warmup. Consequently, low device-task perturbation still does not imply low end-to-end collection cost. Registers, complete schedule perturbation and native-versus-off device cost remain unmeasured here. We retain the known compile failures, high-cost regimes and earlier contended NVIDIA results; clean-window NVIDIA performance remains deferred. None is counted as a passed cost or accuracy gate.

6.3 Profiling-Guided Operator Optimization

We replay the two diagnoses through the public service, distinguishing a retrospective diagnostic replay from first discovery of an optimization. The controlled recovery measurements retain their original source/runtime versions; replaying a diagnostic does not turn them into final-source performance measurements.

Worked request selection. This retrospective GCU example starts from the retained exact-specialization inventory. Filtering `debug_tracked_table` for `tt.store` identifies site 29; the public request selects it, program 0 and capacity 64 with scalar publication. `ai/execution.json` exposes organization; `ai/observations.json` exposes association, loss and clock status. The four- and one-item binaries have different hashes: replay checks each inventory separately despite their identical numeric store IDs. Each retains one associated record without loss. Public documentation supplies the hardware-test command, output retention and field walkthrough. This does not measure onboarding or automatic discovery; capacity does not bound instrumentation or compilation cost.

Public evidence	Decision and independent check
GCU ownership and scratch exchange	Try four work-items $\rightarrow$ one; three native batches yield 1.615–1.621$\times$. Two-item control rules out barrier presence alone; exclusive barrier cost is unavailable.
AIC signal counts 128$\rightarrow$16; AIV wait decreases in both modes	Try $B_K = 256$; retained native time 310.27$\rightarrow$164.12 μs. Local wait magnitudes are mode-sensitive, not uninstrumented costs.
Logical programs 256$\rightarrow$64 and lower source-requested loading	Try larger output tile; native time 164.12$\rightarrow$58.45 μs. Further wait reduction is not stable; individual causal shares are unknown.

Table 2: Retrospective evidence-to-decision replay. Historical recovery runs retain native collection with source observers disabled. Final-service evidence does not require the Study's conditional absolute placement.

Cooperative Softmax. Two bounded public captures use the canonical 127×769 inputs, one row per program, grid 127, and a request for program 0 and one compiler-identified store site. Each produces one correctly associated dynamic record and passes CPU-reference, input-immutability and output-canary checks. The compiled execution manifest, rather than the store interval, supplies the decisive information: four work-items own contiguous useful-column counts 256/256/256/1 and exchange two reduction partials through two shared scratch buffers and two barriers. A one-item request owns all 769 useful columns and removes the scratch exchange. Ten shared-storage IR sites in the former manifest are allocation/store/view/load/deallocation operations for those two buffers, not ten physical messages.

This output suggests testing fewer cooperating work-items while holding math, input, grid, and row grouping fixed. The retained source-observer-disabled three 30-call batches yield 1.615–1.621$\times$ improvement for four to one; the two-item control retains both barriers yet approaches the one-item latency. The diagnosis therefore concerns the cooperative organization, not a measured exclusive barrier cost. The replay uses the same mathematical-function AST and canonical input bytes as those controls, but a separately recorded tool version. The service reproduces the structural clue; its unknown-frequency GCU ticks do not provide a microsecond breakdown of the recovery.

Producer–consumer fusion. The public bundle identifies one AIC producer and two AIV consumers and retains the same-compilation typed readiness channel. Its selected dynamic records identify the consumer's long initial readiness bracket, while the producer's repeated signal and checkpoint counts expose the change from 128 to 16 producer steps when only B_K increases. These facts motivate testing producer granularity; the direction survives both observation modes in Table 1. The retained source-observer-disabled intervention decreases task time from 310.27 to 164.12 μs. The next output-tile change decreases logical instance count from 256 to 64 and source-requested loading; its additional recovery to 58.45 μs does not require claiming lower local waiting. This is the two-stage 5.31$\times$ recovery of the study, with its original independent timing captures.

This replay reproduces a useful decision through actor, dependency and occurrence information; it does not reproduce the study's bespoke dynamic buffer/iteration identities or certify its conditional shared-clock plot. The decision needs neither a cross-actor absolute timeline nor precise uninstrumented wait costs. The service's contribution is the supported execution explanation with explicit evidence and cost boundaries, rather than automatic causal optimization.

6.4 Mechanism Checks

The publication repair provides a concrete ownership counterexample: the original GCU vector record store preserves mathematical results yet loses words when only one work-item publishes. Complete scalar publication and the explicit complete-record-per-work-item alternative pass the byte and tag oracles. Publisher selection alone is therefore insufficient.

For readiness, the original LLVM manifest retains encoded primitives without inventing peers. The same-source typed SDK capture supplies the static producer–consumer channel; print-only replay preserves actor LLVM bodies. For reconstruction, repeated calls, different specializations and two actual streams test invocation isolation. Ambiguous identities and unsupported clocks remain unassociated in the negative contracts. These checks validate concrete mechanism boundaries, not a comparative tool benchmark or measured backend extension effort.

6.5 Scope

The evaluation covers the three available devices and their pinned software stacks. Other adapters, production deployment, and distributed workloads are not validated here.

7 Engineering Experience and Scope

The operational setting is development and controlled qualification on three research machines, using the existing developer Docker containers and pinned compiler/SDK stacks. We report integration failures and diagnostic workflows, not production adoption, application speedups, or a user survey. The central question is which apparently reasonable profiling assumptions failed and what evidence a developer needs before using their output.

7.1 Lessons from Backend Integration

The logger has its own execution mapping. A mathematically correct kernel can publish an invalid trace. An early GCU logger combined a single selected writer with a vector store whose lowering distributed record words across work-items. Selecting one work-item left part of the record unwritten. Inspecting the lowered stores exposed the mismatch. In the retained failed four-item capture, 59 records have only their first four 32-bit words written. The first record is $[7,8,1,0,0,\ldots,0]$: its timestamp fields are unwritten, not evidence of zero execution time. The nonzero-counter oracle fails despite correct math. The repair changes the logger's ownership: scalar words are published by the selected work-item, or the complete logical record is first made available in each work-item's ownership slice. These alternatives have different costs. Trace validation must cover the actual writers and committed record bytes as well as numerical outputs. A reusable check is to run the same known-count kernel with one and several cooperating work-items, then verify every committed record word and tag. Testing only the one-item case would miss this failure. Passing the check establishes publication correctness; it does not establish accurate interval costs.

Association must survive runtime boundaries. The Ascend launch caller and SDK worker can be different threads; caller-side last-task queries therefore do not establish the launched task's identity. The retained worker receipt crosses this boundary. A second boundary appears inside the native output: a mixed dispatch has both dispatch and compute context views. Their shared stream/task fields are insufficient to choose one view. Context membership and the SDK compute-task reference distinguish them while retaining both originals. Finally, temporary SDK output must outlive native finalization so Python can construct and retain this evidence. In a retained four-launch mixed capture, the verified import yields four canonical compute tasks and retains four dispatch-context views separately; the latter do not become additional source-kernel costs. Counting all eight views as compute would report eight tasks for four launches. This is an offline counterexample from retained views, not a measured legacy join or another hardware experiment. The resulting invariant is stronger than a matching task number: the launch receipt, worker query, capture lifetime, and unique compute membership must agree. A reusable test interleaves repeated and different specializations on two streams, and separately removes or duplicates a receipt/context reference. The first case

must retain the correct source binding; the latter must remain unassociated. Missing identity evidence therefore changes what the developer can infer, rather than silently selecting the nearest native event.

Bounded recording is not a bound on perturbation. Small output and selected instances do not establish low cost. GCU all-site Softmax instrumentation initially took about 4.65× the native device task time. The later one-store scalar experiment has much lower device perturbation, but detailed host submission remains about 5.2 ms. NVIDIA site filtering and Ascend program publication predicates also have distinct measurement boundaries: discarded output does not imply that counter reads or instrumentation were absent. On Ascend, compiler selection halves the small RMSNorm record count, yet device perturbation remains 60–78%. The reusable validation is paired measurement of the unchanged workload and each actual observation mode, keeping device, host, export, and storage costs separate. When the cost gate fails, a smaller trace cannot certify a representative local interval. The supported operating regime is selective offline diagnosis, with the restrictions in Section 6.

7.2 Scope and Threats to Validity

Device comparisons include compiler/SDK differences, not microarchitecture alone. Finite operator shapes and one held-out family do not establish application speedups or arbitrary compiler-transform support. Partial origins, actor-local clocks and independent replay scopes retain their limits. NVIDIA functional tests do not resolve its deferred performance gate. Association and complete records are necessary but insufficient for faithful local timing: original fusion waits change by 77–83% between modes despite low whole-task perturbation. Only the first intervention's qualitative direction survives; the second relies on cardinality and independent timing.

8 Related Work

Compiler and instruction-level profiling. KPerfIR [2] embeds programmable profiling in the compiler pipeline and supports fine-grained intra-kernel analysis. Neutrino [3] provides programmable probing. Proton [9] combines vendor and instrumentation backends with selective regions and custom metrics. These systems provide foundations for this work; compiler insertion, circular recording, selectable metrics, and multiple vendor backends are not by themselves our contributions.

KPerfIR already relates compiler regions, iterations, and asynchronous behavior; these are not distinguishing capabilities in the abstract. Its Section 5.2 uses backend-specific collaborative publication on AMD, and Section 5.3 analyzes instrumentation-distorted asynchronous waits with conditional replay correction. We reuse the premise that logging and timing require execution semantics. Our GCU counterexample concerns a sole selected writer that owns only four of sixteen record words after vector lowering. Our Ascend controls expose mode-dependent wait redistribution but provide no improved general replay correction: they preserve only a qualitative decision. Retained worker receipts and compute-context membership address a separate invocation boundary in the evaluated SDK. These are concrete implementation and qualification results, rather than first discoveries of collaborative logging or wait distortion. Our implementation examines concrete boundaries encountered in the evaluated compiler and SDK stacks. It validates full-record publication under GCU work-item ownership, extracts typed AIC/AIV readiness from the same SDK compilation, and associates CANN source invocations through actual worker receipts and SDK context membership. These are the concrete integration mechanisms evaluated here; we do not claim the first compiler-assisted profiler, semantic normalization, or multi-vendor interface. Nor do we infer that earlier tools provide only coarse counters.

Context and accelerator analysis. DeepContext [8] connects program contexts across the deep learning stack, and PASTA [5] supplies modular accelerator analysis. PASTA normalizes event semantics and timing metadata while retaining vendor-specific handlers. Event normalization and backend extensions therefore also form part of the prior infrastructure; our comparison must identify the additional supported compiler mappings and observation-association mechanisms. Broader workload-context analysis complements this scope: our validation addresses compiled operator observations and their attribution boundaries, not complete application or distributed-training diagnosis.

9 Conclusion

FlagPrism connects compiled execution organization, typed dynamic observations and actual invocation evidence for selective operator diagnosis on three backend stacks. Two retrospective workflows reproduce useful structural and qualitative optimization decisions with independent recovery measurements. The most useful systems lessons concern the logger's own ownership, worker/context identity, and the separation of record correctness, local fidelity and collection cost. Partial origins, mode-sensitive waits, expensive collection and unverified shared clocks remain visible limits of the implemented service.

References

[1] FlagOS. FlagTree: A unified compiler for multiple ai chip backends. Project repository. Accessed October 12, 2026. https://github.com/flagos-ai/FlagTree.

[2] Yue Guan, Yuanwei Fang, Keren Zhou, Corbin Robeck, Manman Ren, Zhongkai Yu, Yufei Ding, and Adnan Aziz. KPerfIR: Towards a open and compiler-centric ecosystem for GPU kernel performance tooling on modern AI workloads. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 205–220. USENIX Association, 2025. https://www.usenix.org/conference/osdi25/presentation/guan.

[3] Songlin Huang and Chenshu Wu. Neutrino: Fine-grained GPU kernel profiling via programmable probing. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 331–355. USENIX Association, 2025. https://www.usenix.org/conference/osdi25/presentation/huang-songlin.

[4] Huawei. AI Core architecture. Ascend CANN documentation. Accessed October 12, 2026. https://www.hiascend.com/doc_center/source/en/canncommercial/800/opdevg/tbeaicpudevg/atlasopdev_10_0008.html.

[5] Mao Lin, Hyeran Jeon, and Keren Zhou. PASTA: A modular program analysis tool framework for accelerators. In *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2026. https://2026.cgo.org/details/cgo-2026-papers/28/PASTA-A-Modular-Program-Analysis-Tool-Framework-for-Accelerators.

[6] Moore Threads. MUSA hardware architecture. MUSA SDK 4.3.x programming guide. Historical version; architecture statements apply to its documented devices. Accessed October 12, 2026. https://docs.mthreads.com/musa-sdk/version-4.3.x/programming_guide/Chapter02/.

[7] NVIDIA. Hopper tuning guide. CUDA documentation. Accessed October 12, 2026. https://docs.nvidia.com/cuda/archive/13.2.0/hopper-tuning-guide/index.html.

[8] Qidong Zhao, Hao Wu, Yueming Hao, Zilingfeng Ye, Jiajia Li, Xu Liu, and Keren Zhou. Deepcontext: A context-aware, cross-platform, and cross-framework tool for performance profiling and analysis of deep learning workloads. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 48–63, 2025. https://arxiv.org/abs/2411.02797.

[9] Keren Zhou, Tianle Zhong, Hao Wu, Jihyeong Lee, Yue Guan, Yufei Ding, Corbin Robeck, Yuanwei Fang, Jeff Niu, and Philippe Tillet. Proton: Towards multi-level, adaptive profiling for triton. In *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2026. https://2026.cgo.org/details/cgo-2026-papers/23/Proton-Towards-Multi-level-Adaptive-Profiling-for-Triton.